



Publish panel

Reuse the publish-to-folder workflow across different entity types.

Problem and boundary

Public contract

Working integration

@epam/ai-dial-publish-panel



When this library is useful

Reuse the publish-to-folder workflow across different entity types.

01

Choose a destination

Search and select a publication folder.

02

Explain history

Show earlier publications before replacing or versioning.

03

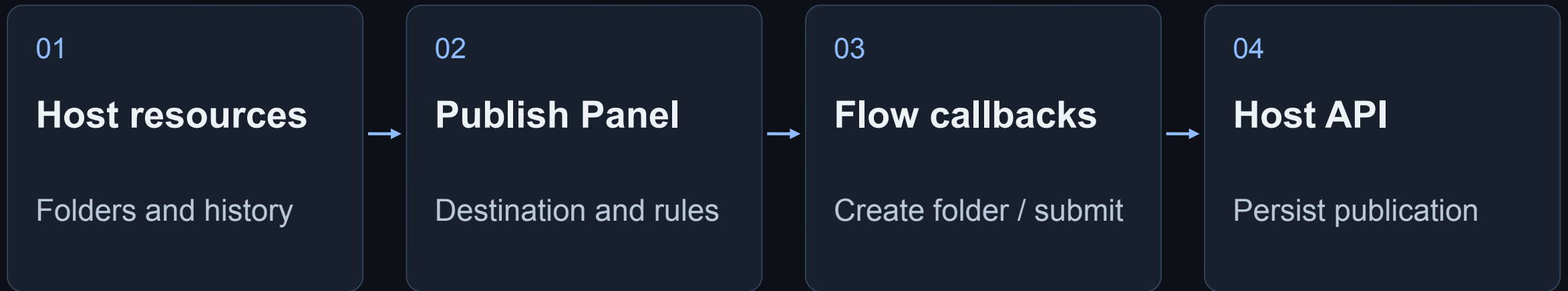
Share one workflow

Use the same panel for deployments and conversations.



Follow the ownership boundary

Inputs and effects stay explicit.



Library: Folder/history UI and a generic publish-flow state helper.



The public contract to start with

PublishPanel · selected surface; full API index is in the bundle.

Input or operation	Contract
folderItems	PublishFolderNode[]
onCreateFolder	(parentPath: string[], name: string) => Promise<void>
rules	PublicationRule[]
hasWriteAccess	boolean



A minimal workspace integration

TypeScript / React example · imports and host setup abridged.

```
function PublishDemo(  
  host: Omit<PublishPanelProps, 'resource'>,  
) {  
  return <PublishPanel  
    {...host}  
    resource={{ title: 'Demo planning notes' }}  
  />;  
}
```

Full example: [examples/publish-panel.tsx](#) · uses [@epam/ai-dial-publish-panel](#)



Wire a realistic host workflow

Keep publish panel focused on its public responsibility.

01

Prepare the resource

Supply the resource title, labels and publication controls.

02

Collect the request

The panel gathers the publication choices.

03

Publish through the host

Run the host callback and reflect its pending or error state.



Available customization

01

Generic summary

resource and
renderSummary avoid
requiring a catalog-
specific model.

02

State helper

usePublishFlow
coordinates selection
and submission state.

03

Access-rule controls

Author, rules and rule-
source options are
explicit inputs.



Find the integration and its checks

Existing tests are evidence of intended behavior; suite results are not claimed.

Evidence	Where to look
Real consumer	<code>chat-hooks/src/conversation/deriveConversationRowActionState/deriveConversationRowActionState.ts</code>
Test evidence	converts folder nodes into a rooted, folder-only file-manager tree
Test evidence	renders the empty-state message when there are no entries



Constraints that affect integration

01

Permission truth

`hasWriteAccess` must come from a host authorization decision.

02

Folder mutations

`onCreateFolder` is asynchronous and can fail.

03

Version semantics

The host decides whether replacement or a new version is appropriate.